Prof. Dr. Holger Schmidt – Professor for IT Security, Computer Science
Faculty of Computer Science
University of Applied Sciences and Arts Dortmund

Fachhochschule Dortmund
University of Applied Sciences and Arts

Requirements "Crypto Gym"

The goal is to implement a *Crypto Gym (CG)* software that allows users to experiment with individual cryptographic algorithms on ASCII text input. Each operation applies **exactly one** cryptographic primitive at a time. The GUI is designed as an interactive laboratory where all intermediate values are visible, editable, and manipulable.

Functional Requirements

GUI Layout

CG MUST provide a graphical user interface as follows:

- **Plaintext field:** An editable text area for ASCII text.
- **Hex fields:** Editable text areas for `ciphertext`, `key/key pair`, `IV/nonce`, `salt`, `hash`, `MAC`, and `signature`. The user MUST be able to manipulate any hex value (e. g., alter a ciphertext, change an IV, modify a key) and re-run the operation to observe the effect.
- **Result / Status field:** A read-only area for displaying operation results (e. g., `valid` / `invalid`) and error messages. CG MUST NOT crash on invalid hex input, wrong key lengths, corrupted ciphertexts, etc.; instead, it MUST display a meaningful error message in this field.

Length Information

The following length information MUST be displayed, expressed in **raw bytes**:

- Length of the plaintext
- Length of the ciphertext
- Length of the key
- Length of the IV/nonce
- Length of the hash / MAC / signature

Cryptographic Operations

The user MUST be able to select exactly one of the following operations at a time. Each operation MUST be independent; combinations of operations are not supported.

a) **Symmetric Encryption**
   - Algorithm: `AES`, 256-bit key
   - Block modes: `CBC`, `OFB`, `GCM`

- Padding schemes: `NoPadding`, `PKCS7Padding`, `CTS`[1]
- Input: Plaintext (ASCII), Key (hex), IV/nonce (hex, auto-generated if empty)
- Output: Ciphertext (hex)

b) **Symmetric Decryption**

- Same algorithm, mode, and padding options as **Symmetric Encryption**
- Input: Ciphertext (hex), Key (hex), IV/nonce (hex)
- Output: Plaintext (ASCII)

c) **Password-Based Encryption**

- `AES`, 256-bit key, KDF: `Argon2id`
- `ChaCha20`, 256-bit key, KDF: `Argon2id`
- Argon2id parameters: $m = 64\,\text{MiB}$, $t = 3$, $p = 4$
- Input: Plaintext (ASCII), Password (text), Salt (hex, auto-generated if empty)
- Output: Ciphertext (hex)

d) **Password-Based Decryption**

- Same algorithm and KDF options as **Password-Based Encryption**
- Input: Ciphertext (hex), Password (text), Salt (hex)
- Output: Plaintext (ASCII)

e) **Hashing**

- `SHA-256`
- Input: Plaintext (ASCII)
- Output: Hash digest (hex)

f) **MAC Generation**

- `AES-CMAC`
- `HMAC-SHA256`
- Input: Plaintext (ASCII), Key (hex)
- Output: MAC (hex)

g) **MAC Verification**

- Same algorithm options as **MAC Generation**
- Input: Plaintext (ASCII), Key (hex), MAC (hex)

[1] CTS (Ciphertext Stealing) is a padding scheme, not a block mode.

- Output: Boolean (`valid` / `invalid`)

h) **Digital Signature Creation**

- `ECDSA`, curve `P-256`
- Input: Plaintext (ASCII), Private key (hex)
- Output: Signature (hex)

i) **Digital Signature Verification**

- `ECDSA`, curve `P-256`
- Input: Plaintext (ASCII), Public key (hex), Signature (hex)
- Output: Boolean (`valid` / `invalid`)

Key Management

CG MUST be able to generate and display cryptographic keys in hexadecimal. The user MUST be able to either generate a new key or paste an existing one. For ECDSA, CG MUST generate and display a key pair (private and public key).

Persistence

CG MUST be able to save and load the following data:

- Keys (symmetric keys, private keys, public keys)
- IVs/nonces/salts
- Ciphertexts
- Hash digests
- MACs
- Signatures
- Corresponding metadata (algorithm, mode, padding, parameters)

All binary values MUST be stored as **hexadecimal strings**. The storage format MUST be a structured file format, preferably JSON[2]. Each saved entity MUST be stored separately (i. e., one file per key, one file per result, etc.) or in a single structured file with clearly separated sections.

[2] https://github.com/google/gson, accessed on 2026-10-05

Non-Functional Requirements

CG SHOULD be implemented using Java and MUST use the latest version of `Bouncy Castle`[3]. The software architecture MUST allow extensibility so that additional cryptographic algorithms can be easily added.
CG's source code MUST be documented. In case Java is used for implementation, `Javadoc` and `Doxygen`[4] MUST be used for documentation. CG MUST be validated based on unit testing. In case Java is used for implementation, `JUnit`[5] MUST be used for testing. Moreover, an analysis of the test coverage in terms of statement and branch coverage MUST be carried out using a tool such as `JaCoCo`[6].
Ideally you will work with `IntelliJ IDEA`[7], `Visual Studio Code`[8], or `Eclipse`[9], `Git`[10] and `Maven`[11]. `IntelliJ IDEA Ultimate` as part of the `JetBrains Educational Pack` is available for free *for educational use only* via registration on https://www.jetbrains.com/shop/eform/students.

AI Usage Policy

The use of AI-based tools (e. g., large language models, code completion assistants) is subject to the following restrictions:

Permitted Use

AI tools MAY be used for the following purposes:

- **GUI implementation:** Layout, event handling, bindings, and other user-interface code.
- **Consultation:** As a sparring partner for architectural decisions, design questions, or general guidance. The student MUST be able to explain and justify any AI-suggested design decision independently.
- **Code documentation:** Generating or refining `Javadoc` and `Doxygen` comments.
- **Test code:** Writing `JUnit` test cases.

[3] https://www.bouncycastle.org, accessed on 2026-10-05
[4] https://www.stack.nl/~dimitri/doxygen/, accessed on 2026-10-05
[5] https://junit.org/, accessed on 2026-10-05
[6] https://www.eclemma.org/jacoco/, accessed on 2026-10-05
[7] https://www.jetbrains.com/idea/, accessed on 2026-10-05
[8] https://github.com/microsoft/vscode, accessed on 2026-10-05
[9] https://eclipse.org/, accessed on 2026-10-05
[10] https://git-scm.com/, accessed on 2026-10-05
[11] https://maven.apache.org/, accessed on 2026-10-05

Prohibited Use

AI tools MUST **not** be used to generate the following:

- Cryptographic core logic (algorithm invocation, key management, IV/nonce handling, hex conversion, padding/mode configuration)
- Persistence logic (serialization, deserialization, file I/O)
- Any other code not explicitly listed under "Permitted Use"

All code that is not explicitly permitted for AI generation MUST be written by the student independently. The student MUST be able to explain every line of their own code during an oral examination or code review.

General AI Guideline

In addition to the above, the document *"AI Guideline for written, unsupervised Examinations"* provided via ILIAS applies to this project. In case of conflict between this section and the general guideline, the more restrictive rule applies.

Specs Grading Form

The software project is graded based on the specs grading form provided via ILIAS. The form provides information about which requirements must be fulfilled in order to receive a certain grade.